

Documentation Technique

Projet InsertionSQL - JavaFX + MySQL

Guide complet pour comprendre et reproduire le projet

Ce document explique en detail le projet InsertionSQL, une application JavaFX qui permet de se connecter a une base de donnees MySQL distante (dans une VM VirtualBox) et d'interagir avec les donnees via une interface graphique.

Technologie	Version	Role
Java	24	Langage de programmation principal
JavaFX	21.0.6	Interface graphique
MySQL	8.0.45	Base de donnees
Maven	3.x	Gestionnaire de dependances
VirtualBox	7.x	Machine virtuelle Ubuntu pour le serveur
IntelliJ IDEA	2025	IDE de developpement

Sommaire

Chapitre	Contenu
1	Le pattern MVC - Theorie et application
2	Structure du projet et packages
3	La base de donnees cinefilm
4	Configuration VirtualBox et reseau
5	Explication de chaque fichier Java
6	Les fichiers FXML - Interface graphique
7	La connexion a la base de donnees
8	Erreurs rencontrees et solutions
9	Comment reproduire le projet

Chapitre 1 : Le pattern MVC

1.1 Qu'est-ce que le MVC ?

MVC signifie Modele - Vue - Controleur. C'est un patron de conception (design pattern) qui organise le code en 3 parties distinctes pour rendre l'application plus facile a maintenir et a faire evoluer.

Partie	Role	Exemple dans InsertionSQL
Modele (M)	Represente les donnees et la logique metier. Fait les requetes SQL. Ne sait pas comment les donnees sont affichees.	Film.java, Critique.java, Notation.java, DatabaseManager.java
Vue (V)	Represente l'interface graphique. Ne contient aucune logique. Affiche ce que le controleur lui donne.	hello-view.fxml, main-view.fxml
Controleur (C)	Fait le lien entre le Modele et la Vue. Reçoit les actions de l'utilisateur et met a jour la Vue.	LoginController.java, MainController.java

1.2 Schema de fonctionnement

Utilisateur clique sur un bouton

|

v

Controleur (reçoit l'action)

|

| demande les donnees

v

Modele (interroge MySQL)

|

| retourne les donnees

v

Controleur (met a jour la Vue)

|

v

Vue (affiche les donnees)

1.3 Pourquoi utiliser le MVC ?

- Separation des responsabilites : chaque classe a un role precis
- Maintenance facilitee : modifier la Vue ne change pas le Modele
- Reutilisabilite : le Modele peut etre utilise par plusieurs Vues
- Lisibilite : le code est plus organise et plus facile a comprendre

■ Dans notre projet, le MainController fait encore un peu trop de choses. Dans un MVC ideal, les requetes SQL seraient uniquement dans les classes Modele.

Chapitre 2 : Structure du projet et packages

2.1 Organisation des packages

Le projet est organise en 3 packages qui correspondent aux 3 couches du MVC :

```
com.example.insertionsql/  
■■■■ model/ <- Couche Modele  
■ ■■■■ DatabaseManager.java <- Gestion de la connexion MySQL  
■ ■■■■ Film.java <- Modele de la table film  
■ ■■■■ Critique.java <- Modele de la table critique  
■ ■■■■ Notation.java <- Modele de la table notation  
■■■■ controller/ <- Couche Controleur  
■ ■■■■ LoginController.java <- Gere la page de connexion  
■ ■■■■ MainController.java <- Gere la page principale  
■■■■ HelloApplication.java <- Point d'entree de l'application  
■■■■ Launcher.java <- Lance l'application  
■■■■ resources/  
■■■■ com/example/insertionsql/  
■■■■ hello-view.fxml <- Vue de la page connexion  
■■■■ main-view.fxml <- Vue de la page principale
```

2.2 Le fichier pom.xml

Le pom.xml est le fichier de configuration Maven. Il contient les dependances necessaires au projet :

Dependance	Version	Role
javafx-controls	21.0.6	Composants JavaFX (TableView, Button, Label...)
javafx-fxml	21.0.6	Support des fichiers FXML
mysql-connector-j	9.6.0	Driver JDBC pour se connecter a MySQL

■ La dependance *mysql-connector-j* doit etre placee **DANS** la balise du *pom.xml*. Sinon Java ne trouve pas le driver MySQL.

Chapitre 3 : La base de donnees cinefilm

3.1 Structure de la base de donnees

La base de donnees cinefilm contient 3 tables :

Table	Colonnes	Description
film	fID, title, year, director	Stocke les films
critique	cID, name	Stocke les utilisateurs/critiques
notation	cID, fID, stars, ratingDate	Stocke les notes donnees par les critiques

3.2 Schema relationnel

```
film (fID PK, title, year, director)
|
| fID (cle etrangere)
|
notation (cID FK, fID FK, stars, ratingDate)
|
| cID (cle etrangere)
|
critique (cID PK, name)
```

3.3 Commandes SQL pour creer la base

```
CREATE DATABASE cinefilm;
USE cinefilm;
CREATE TABLE film (
fID INT AUTO_INCREMENT PRIMARY KEY,
title VARCHAR(100) NOT NULL,
year INT,
director VARCHAR(100)
);
CREATE TABLE critique (
cID INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(100) NOT NULL
);
CREATE TABLE notation (
cID INT,
fID INT,
stars INT CHECK (stars BETWEEN 1 AND 5),
ratingDate DATE,
FOREIGN KEY (fID) REFERENCES film(fID),
FOREIGN KEY (cID) REFERENCES critique(cID)
);
```

Chapitre 4 : Configuration VirtualBox et reseau

4.1 Pourquoi VirtualBox ?

Le serveur MySQL tourne dans une machine virtuelle Ubuntu. Pour y acceder depuis Windows, on utilise la redirection de ports de VirtualBox qui fait le lien entre ton PC Windows et la VM Ubuntu.

4.2 Schema reseau

```
PC Windows (IntelliJ)
|
| connexion sur 127.0.0.1:3306
v
VirtualBox (redirection de port)
| Port hote 3306 -> Port invite 3306
v
VM Ubuntu (10.0.2.15)
|
| MySQL ecoute sur 0.0.0.0:3306
v
Base de donnees cinefilm
```

4.3 Regles de redirection de ports

Nom	Protocole	Port hote	Port invite	Utilisation
MySQL	TCP	3306	3306	Connexion JDBC depuis Java
PHP	TCP	8000	80	Acces au site web PHP
SSH	TCP	2222	22	Connexion SSH pour transfert de fichiers

4.4 Configuration MySQL pour les connexions exterieures

Dans le fichier /etc/mysql/mysql.conf.d/mysqld.cnf de la VM :

```
bind-address = 0.0.0.0 # Accepte toutes les connexions
mysqlx-bind-address = 0.0.0.0 # Idem pour le protocole X
```

Puis donner les droits a root depuis n'importe quelle IP :

```
CREATE USER 'root'@'%' IDENTIFIED BY 'root';
GRANT ALL PRIVILEGES ON *.* TO 'root'@'%' WITH GRANT OPTION;
FLUSH PRIVILEGES;
```

■ La VM doit etre eteinte quand on configure la redirection de ports dans VirtualBox, sinon les changements ne sont pas pris en compte.

Chapitre 5 : Explication de chaque fichier Java

5.1 DatabaseManager.java (model)

Ce fichier centralise la configuration de connexion a MySQL. Il stocke l'IP, l'utilisateur, le mot de passe et le nom de la BDD, et fournit une methode getConnection() utilisee par tous les autres fichiers.

```
public class DatabaseManager {
    private static String ip, user, password, database;
    // Appele une seule fois apres la connexion reussie
    public static void setConfig(String ip, String user,
    String password, String database) { ... }
    // Appele a chaque fois qu'on a besoin d'une connexion
    public static Connection getConnection() throws SQLException { ... }
}
```

5.2 LoginController.java (controller)

Gere la page de connexion. Quand l'utilisateur clique sur 'Se connecter', il teste la connexion MySQL avec les infos saisies. Si ca marche, il sauvegarde la config dans DatabaseManager et charge main-view.fxml.

```
@FXML
public void seConnecter() {
    // 1. Recupere les valeurs des champs
    // 2. Teste la connexion MySQL
    // 3. Si OK -> sauvegarde config et charge main-view.fxml
    // 4. Si erreur -> affiche message en rouge
}
```

5.3 MainController.java (controller)

Gere la page principale. Il fait 3 choses principales :

- Afficher le contenu d'une table quand on clique sur un bouton du menu
- Charger dynamiquement les colonnes d'une table dans des ComboBox
- Insérer une nouvelle ligne dans la table selectionnee

Methode	Role
initialize()	Charge la liste des tables dans le ComboBox au demarrage
afficherTable()	Execute SELECT * FROM table et remplit le TableView
afficherCritique/Film/Notation()	Raccourcis qui appellent afficherTable()
chargerColonnes()	Cree un ComboBox par colonne de la table selectionnee
insérer()	Execute INSERT INTO avec les valeurs saisies dans les ComboBox

5.4 Film.java, Critique.java, Notation.java (model)

Ces classes representent les tables de la base de donnees. Chaque attribut correspond a une colonne de la table. Elles contiennent aussi des methodes statiques pour interagir avec la BDD :

Methode	Description
---------	-------------

getAll()	Execute SELECT * FROM table et retourne une liste d'objets
insert(...)	Execute INSERT INTO avec les parametres donnees

■ Dans un MVC ideal, MainController devrait utiliser ces methodes au lieu de faire directement les requetes SQL. C'est une amelioration possible.

Chapitre 6 : Les fichiers FXML

6.1 Qu'est-ce que le FXML ?

FXML est un langage XML utilise par JavaFX pour definir l'interface graphique. C'est l'equivalent du HTML pour les applications de bureau. Il permet de separer la Vue (FXML) du Controleur (Java).

Attribut FXML	Role
fx:controller	Indique quelle classe Java gere ce fichier (le controleur)
fx:id	Donne un identifiant pour acceder au composant depuis le controleur avec @FXML
onAction	Indique quelle methode du controleur appeler quand on clique
prefWidth/Height	Definit la taille preferee du composant
style	CSS inline pour personnaliser l'apparence

6.2 hello-view.fxml - Page de connexion

Cette page contient un formulaire avec 4 champs texte (IP, utilisateur, mot de passe, base de donnees) et un bouton Se connecter. Le controleur associe est LoginController.

```
fx:controller='com.example.insertionsql.controller.LoginController'
Composants principaux :
- txtIP : TextField pour l'adresse IP
- txtUser : TextField pour le nom d'utilisateur
- txtPassword : PasswordField pour le mot de passe (masque les caracteres)
- txtDatabase : TextField pour le nom de la base de donnees
- labelErreur : Label rouge pour afficher les erreurs
- Bouton : onAction='#seConnecter' -> appelle LoginController.seConnecter()
```

6.3 main-view.fxml - Page principale

Cette page est divisee en deux zones avec un HBox :

- Zone gauche (VBox bleu fonce) : menu de navigation + formulaire d'insertion
- Zone droite : TableView pour afficher les donnees

```
Structure :
HBox
■■■ VBox (menu gauche, fond bleu)
■ ■■■ Bouton 'Critiques' -> afficherCritique()
■ ■■■ Bouton 'Films' -> afficherFilm()
■ ■■■ Bouton 'Notations' -> afficherNotation()
■ ■■■ ComboBox cmbTable -> chargerColonnes()
■ ■■■ VBox vboxChamps -> rempli dynamiquement par Java
■ ■■■ Bouton 'Insérer' -> inserer()
■ ■■■ Label labelMessage -> messages de succes/erreur
■■■ VBox (zone droite)
■■■ Label labelTable -> nom de la table affichee
■■■ TableView tableView -> donnees de la table
```

Chapitre 7 : La connexion a la base de donnees

7.1 Le driver JDBC

JDBC (Java Database Connectivity) est l'API Java pour se connecter aux bases de donnees. Pour MySQL, on utilise le driver mysql-connector-j qui doit etre ajoute dans le pom.xml.

Le driver doit etre charge explicitement avec :

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

7.2 L'URL de connexion

L'URL JDBC contient toutes les informations de connexion :

```
jdbc:mysql://127.0.0.1:3306/cinefilm?useSSL=false&allowPublicKeyRetrieval=true
```

Partie	Valeur	Description
jdbc:mysql://	jdbc:mysql://	Protocole JDBC pour MySQL
IP	127.0.0.1	Adresse du serveur (localhost via VirtualBox)
:3306	:3306	Port MySQL par defaut
/cinefilm	/cinefilm	Nom de la base de donnees
useSSL=false	false	Desactive SSL (non necessaire en local)
allowPublicKeyRetrieval	true	Permet la recuperation de la cle publique

7.3 Le flux de connexion dans l'application

1. Utilisateur saisit IP, user, password, database dans hello-view.fxml
2. LoginController.seConnecter() teste la connexion
3. Si OK -> DatabaseManager.setConfig() sauvegarde les infos
4. main-view.fxml est charge
5. MainController.initialize() appelle DatabaseManager.getConnection()
6. Chaque action (afficher, inserer) appelle DatabaseManager.getConnection()
7. La connexion est fermee apres chaque operation (conn.close())

■ Il est important de fermer la connexion apres chaque operation avec `conn.close()` pour eviter les fuites de connexions.

Chapitre 8 : Erreurs rencontrees et solutions

Erreur	Cause	Solution
Communications link failure	Port 3306 inaccessible depuis Windows	Verifier redirection VirtualBox, bind-address=0.0.0.0, et relancer apres avoir eteint la VM
No suitable driver found	Driver MySQL absent ou mal configure	Ajouter mysql-connector-j dans du pom.xml et faire Ctrl+Shift+O
Access denied for user root	Mauvais mot de passe ou utilisateur non autorise depuis l'exterieur	Creer l'utilisateur root@'%' avec GRANT ALL PRIVILEGES
Location is not set	Chemin vers le fichier FXML incorrect apres reorganisation en packages	Utiliser le chemin absolu : /com/example/insertionsql/main-view.fxml
Cannot resolve symbol VBox	Import manquant dans le fichier Java	Ajouter : import javafx.scene.layout.VBox;
package java.sql is not visible	module-info.java ne declare pas java.sql	Ajouter 'requires java.sql;' dans module-info.java ou supprimer ce fichier
class Client is public, should be declared in Client.java	Nom du fichier en minuscule (client.java) ne correspond pas au nom de la classe	Renommer via Refactor -> Rename en respectant la casse
TcpTestSucceeded : False	La VM etait allumee lors de la configuration de la redirection VirtualBox	Eteindre la VM, configurer la redirection, puis rallumer la VM

Chapitre 9 : Comment reproduire le projet

Etape 1 : Preparer la VM Ubuntu

Dans VirtualBox, creer une VM Ubuntu et installer :

```
sudo apt update && sudo apt upgrade -y
sudo apt install mysql-server -y
# Configurer MySQL pour les connexions exterieures
sudo nano /etc/mysql/mysql.conf.d/mysqld.cnf
# Changer : bind-address = 0.0.0.0
# Changer : mysqlx-bind-address = 0.0.0.0
sudo systemctl restart mysql
# Autoriser root depuis l'exterieur
sudo mysql -u root -p
ALTER USER 'root'@'localhost' IDENTIFIED WITH mysql_native_password BY 'root';
CREATE USER 'root'@'%' IDENTIFIED BY 'root';
GRANT ALL PRIVILEGES ON *.* TO 'root'@'%' WITH GRANT OPTION;
FLUSH PRIVILEGES;
```

Etape 2 : Configurer VirtualBox

Eteindre la VM, puis dans Configuration -> Reseau -> Redirection de ports :

Nom	Port hote	Port invite
MySQL	3306	3306
SSH	2222	22

Etape 3 : Creer le projet IntelliJ

```
File -> New -> Project -> JavaFX
GroupId : com.example
ArtifactId : InsertionSQL
```

Ajouter dans pom.xml dans la section :

```
<dependency>
<groupId>com.mysql</groupId>
<artifactId>mysql-connector-j</artifactId>
<version>9.6.0</version>
</dependency>
Puis : Ctrl+Shift+O pour synchroniser Maven
```

Etape 4 : Creer la structure de packages

```
Clic droit sur com.example.insertionsql -> New -> Package -> model
Clic droit sur com.example.insertionsql -> New -> Package -> controller
```

Etape 5 : Creer les fichiers dans l'ordre

Ordre	Fichier	Package
1	DatabaseManager.java	model
2	Film.java	model

3	Critique.java	model
4	Notation.java	model
5	LoginController.java	controller
6	MainController.java	controller
7	hello-view.fxml	resources
8	main-view.fxml	resources
9	HelloApplication.java	racine

Etape 6 : Verifier module-info.java

Si le fichier module-info.java existe, ajouter :

```
requires java.sql;
requires com.mysql.cj;
```

Ou supprimer ce fichier si ca cause trop de problemes.

Etape 7 : Tester

Lancer le projet avec le bouton Play. Sur la page de connexion entrer :

Champ	Valeur
Adresse IP	127.0.0.1
Utilisateur	root
Mot de passe	root
Base de donnees	cinofilm

✓ Si tout est bien configure, l'application se connecte et affiche la page principale avec les tables de la base de donnees !